

Punto 2.3 - Relaciones uno a muchos

Ejercicio guiado: configurar explícitamente la relación uno-a-muchos entre `OrdenFabricacion` y `PlanchaAcero`, identificar los extremos principal y dependiente, fijar la clave foránea `OrdenId`, establecer el comportamiento de eliminación y comprobar la carga de la colección con `Include`.

Contexto del proyecto: 2.3 continúa directamente desde 2.2. Las entidades y sus propiedades escalares ya están configuradas. La relación entre orden y plancha ya podía ser descubierta por convención, pero ahora se expresa de forma explícita con Fluent API para que su intención y su comportamiento queden visibles en el modelo. Este punto se limita a la relación `OrdenFabricacion 1 -> N PlanchaAcero`: las relaciones uno-a-uno se estudian en 2.4 y la relación muchos-a-muchos se reserva para 2.5.

Objetivos de aprendizaje

- Comprender qué representa una relación uno-a-muchos.
- Identificar el extremo principal y el extremo dependiente.
- Localizar la clave foránea en la entidad dependiente.
- Configurar la relación con `HasOne`, `WithMany` y `HasForeignKey`.
- Configurar `DeleteBehavior.Cascade` de forma explícita.
- Entender por qué `IsRequired()` es coherente con una FK `int` no anulable.
- Diferenciar un cambio de configuración del modelo de un cambio real del esquema SQL.
- Cargar la colección relacionada con `Include`.
- Comprobar la relación con datos reales en SQL Server LocalDB.
- No adelantar `DetalleOrden`, `CertificadoCalidad` ni `OrdenAleacion`.

Paso 1: Abrir el estado autónomo de 2.3

```
cd M02/PROYECTO/2.3
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

El checkpoint 2.3 contiene el resultado completo del ejercicio. Si estás realizando el laboratorio desde 2.2, aplica los cambios siguientes sobre tu copia del punto anterior y genera después la migración incremental.

Paso 2: Revisar los dos extremos de la relación

Las propiedades relevantes son:

```
public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public List<PlanchaAcero> Planchas { get; set; } = new();
}

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    public OrdenFabricacion Orden { get; set; } = null!;
}
```

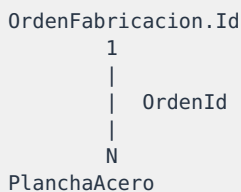
Interpreta el modelo antes de configurarlo:

- `OrdenFabricacion` es el **principal**: una orden puede contener varias planchas.
- `PlanchaAcero` es el **dependiente**: cada plancha pertenece a una orden.
- `OrdenFabricacion.Planchas` es la navegación de colección.
- `PlanchaAcero.Orden` es la navegación de referencia.
- `PlanchaAcero.OrdenId` almacena la clave primaria de la orden asociada.

La clave foránea reside en el dependiente. El nombre `OrdenId` también puede ser reconocido por convención, pero en este punto se declarará explícitamente.

Paso 3: Identificar la convención antes de sustituirla por configuración explícita

EF Core ya puede relacionar estas propiedades:



La colección y la referencia permiten deducir la cardinalidad. La finalidad de 2.3 no es inventar una segunda relación, sino hacer explícita la existente para controlar su comportamiento.

Un nombre como `OrdenFabricacionId` también encaja en las convenciones habituales. Un nombre no convencional, por ejemplo `IdOrden`, requeriría configuración explícita para evitar una FK sombra o una relación distinta de la esperada.

Paso 4: Configurar la relación con Fluent API

En `OnModelCreating`, la configuración efectiva de `PlanchaAcero` queda así:

```

modelBuilder.Entity<PlanchaAcero>(entity =>
{
    entity.ToTable("PlanchasAcero");
    entity.HasKey(x => x.Id);
    entity.Property(x => x.Peso).HasPrecision(18, 3);
    entity.Property(x => x.Activa).HasDefaultValue(true);

    entity.HasOne(x => x.Orden)
        .WithMany(o => o.Planchas)
        .HasForeignKey(x => x.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
});
  
```

Línea a línea:

1. `HasOne(x => x.Orden)` selecciona la navegación de referencia del dependiente.
2. `WithMany(o => o.Planchas)` conecta esa referencia con la colección del principal.
3. `HasForeignKey(x => x.OrdenId)` declara explícitamente qué propiedad actúa como FK.
4. `OnDelete(DeleteBehavior.Cascade)` indica que la eliminación física del principal se propaga a sus planchas.
5. `IsRequired()` expresa que una plancha persistida debe tener una orden válida.

La relación debe configurarse de forma coherente una sola vez. No hace falta repetirla desde `OrdenFabricacion` con otra configuración.

Paso 5: Comprender el comportamiento de eliminación

EF Core permite distintos comportamientos de eliminación. En este ejercicio se utiliza:

```
.onDelete(DeleteBehavior.Cascade)
```

Cascade es coherente con el modelo actual: si una orden se elimina físicamente, sus planchas dependientes no deben quedar huérfanas.

Otros comportamientos existen, pero no se aplican aquí de forma intercambiable:

- **Restrict / NoAction**: impiden la eliminación del principal mientras existan dependientes, según cómo actúe el proveedor.
- **SetNull**: requiere una FK que pueda admitir `NULL`; `OrdenId` es `int` no anulable.
- los comportamientos de cliente afectan a cómo EF Core gestiona entidades rastreadas y no deben confundirse con una regla SQL idéntica en todos los proveedores.

El objetivo es entender por qué se elige `Cascade`, no memorizar una opción para todas las relaciones.

Paso 6: Compilar antes de generar la migración

Después de introducir la configuración:

```
dotnet build AceriaData.sln --configuration Release
```

El build debe terminar sin errores. Si aparece un error relacionado con `WithMany`, `HasForeignKey` o una navegación inexistente, revisa primero que las propiedades de ambos extremos coincidan con el código del paso 2.

Paso 7: Generar la migración M2_2_3 desde el estado 2.2

Si estás reproduciendo el laboratorio sobre una copia de 2.2:

```
dotnet ef migrations add M2_2_3
```

En el checkpoint entregado la migración ya existe:

```
Migrations/20260927204726_M2_2_3.cs
```

No vuelvas a generar una migración con el mismo nombre dentro del checkpoint final. El comando anterior representa el paso que debe realizarse cuando se construye 2.3 incrementalmente desde 2.2.

Paso 8: Revisar por qué M2_2_3 no necesita operaciones SQL nuevas

Abre `Migrations/20260927204726_M2_2_3.cs`. Su contenido relevante es:

```
protected override void Up(MigrationBuilder migrationBuilder)
{
}

protected override void Down(MigrationBuilder migrationBuilder)
{
}
```

Esto no significa que el ejercicio no haya hecho nada. La relación ya estaba presente en el esquema heredado porque las propiedades y navegaciones permitían a EF Core descubrirla por convención. En 2.3 se hace **explícita** la misma forma relacional:

- FK `PlanchasAcero.OrdenId`;
- principal `OrdenesFabricacion.Id`;

- índice sobre `OrdenId`;
- eliminación en cascada.

Como la configuración explícita coincide con el esquema ya representado por el snapshot anterior, no existe un delta SQL adicional que aplicar.

Esta es una diferencia importante: **cambiar cómo se expresa una relación en el modelo no implica necesariamente cambiar la base de datos.**

▮ Paso 9: Aplicar el historial y verificar la relación real

Ejecuta:

```
dotnet ef database update
dotnet ef migrations list
```

La lista debe incluir `M2_2_3`. En SQL Server Object Explorer puedes comprobar en `dbo.PlanchasAcero`:

- la columna `OrdenId`;
- el índice `IX_PlanchasAcero_OrdenId`;
- la FK `FK_PlanchasAcero_OrdenesFabricacion_OrdenId`;
- la referencia a `OrdenesFabricacion(Id)`;
- el comportamiento de eliminación en cascada.

En este punto no debe aparecer una tabla `DetallesOrden` ni `OrdenesAleaciones`: pertenecen a puntos posteriores.

▮ Paso 10: Insertar una orden con una plancha relacionada

El estado final crea el agregado utilizando la navegación de colección:

```
var orden = new OrdenFabricacion
{
    NumeroOrden = "OF-M2-0001",
    Cliente = "Constructora del Norte",
    FechaCreacion = DateTime.UtcNow,
    Estado = "Pendiente",
    Planchas =
    {
        new PlanchaAcero
        {
            Espesor = 10.5,
            Ancho = 1500,
            Largo = 3000,
            Peso = 371.250m,
            Activa = true,
        }
    },
};

context.OrdenesFabricacion.Add(orden);
context.SaveChanges();
```

Al agregar el principal con una plancha en su colección, EF Core mantiene la relación y asigna la FK correspondiente al guardar.

Si se intentase persistir una plancha con una `OrdenId` que no corresponde a una orden existente, SQL Server rechazaría la operación por integridad referencial.

▮ Paso 11: Cargar la colección con Include

La comprobación funcional del punto usa carga eager:

```
var cargada = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .Single(o => o.NumeroOrden == "OF-M2-0001");

global::System.Console.WriteLine(
    $"2.3 OK | {cargada.NumeroOrden} | Planchas: {cargada.Planchas.Count}");
```

La salida esperada del estado base es:

```
2.3 OK | OF-M2-0001 | Planchas: 1
```

`Include(o => o.Planchas)` pide explícitamente que la colección se cargue con la consulta. No debe suponerse que la navegación se cargará automáticamente.

Paso 12: Resolver el reto con dos planchas

Reto: crear otra orden con exactamente dos planchas, recuperarla con `Include` y demostrar que la colección contiene dos elementos.

El checkpoint contiene el bloque pedagógico **RETO 2.3 - DOS PLANCHAS CON INCLUDE**. La parte esencial es:

```
var ordenReto = new OrdenFabricacion
{
    NumeroOrden = "OF-M2-RETO-23",
    Cliente = "Cliente reto 2.3",
    FechaCreacion = DateTime.UtcNow,
    Estado = "Pendiente",
    Planchas =
    {
        new PlanchaAcero
        {
            Espesor = 8,
            Ancho = 1200,
            Largo = 2500,
            Peso = 188.500m,
            Activa = true,
        },
        new PlanchaAcero
        {
            Espesor = 12,
            Ancho = 1500,
            Largo = 3000,
            Peso = 424.125m,
            Activa = true,
        },
    },
};

context.OrdenesFabricacion.Add(ordenReto);
context.SaveChanges();

var retoCargada = context.OrdenesFabricacion
    .AsNoTracking()
    .Include(o => o.Planchas)
    .Single(o => o.NumeroOrden == "OF-M2-RETO-23");

global::System.Console.WriteLine(
    $"Reto 2.3 planchas: {retoCargada.Planchas.Count}");
```

El resultado esperado es:

```
Reto 2.3 planchas: 2
```

El reto amplía la misma relación 1:N; no introduce todavía una relación 1:1 ni N:M.

Errores comunes

Error	Causa	Corrección
La FK no se asocia a la navegación prevista	La propiedad no sigue una convención y no se configuró explícitamente	Usar <code>HasForeignKey(x => x.OrdenId)</code>
Se crea una FK sombra adicional	La navegación y la FK se configuraron de forma incoherente	Relacionar explícitamente <code>Orden</code> , <code>Planchas</code> y <code>OrdenId</code>
Se configura la relación dos veces con opciones diferentes	Se repite desde ambos extremos con reglas contradictorias	Mantener una única configuración coherente
Se usa <code>SetNull</code> con <code>OrdenId</code> no anulable	La FK no puede almacenar <code>NULL</code>	Mantener <code>Cascade</code> en este escenario o rediseñar conscientemente la nulabilidad
La colección no aparece cargada	Se consultó la orden sin <code>Include</code>	Usar <code>Include(o => o.Planchas)</code> cuando se necesita el grafo
Se crea <code>DetalleOrden</code> o <code>OrdenAleacion</code>	Se adelantó contenido de 2.4 o 2.5	Mantener 2.3 exclusivamente en <code>OrdenFabricacion 1 -> N PlanchaAcero</code>
Se usa <code>EnsureCreated()</code>	Se sustituye el historial incremental por creación directa	Mantener el flujo basado en Migrations

Analogía final

La relación se parece a una orden de fabricación y las planchas producidas para ella. Una orden puede agrupar muchas planchas, pero cada plancha conserva una referencia a una sola orden mediante `OrdenId`. La colección `Planchas` funciona como el listado de piezas asociado a la orden; la navegación `Orden` permite recorrer la relación en sentido contrario. Definir la relación explícitamente equivale a dejar escritas las reglas de ese vínculo en el plano del sistema, incluida la política que se aplica si el principal se elimina físicamente.

Resultado esperado

Al finalizar 2.3, `AceriaData` conserva todas las propiedades de 2.2 y hace explícita la relación `OrdenFabricacion 1 -> N PlanchaAcero`. El historial incluye `M2_2_3`, cuya ausencia de operaciones SQL se explica porque la forma relacional ya había sido descubierta por convención. La aplicación termina con `2.3 OK | 0F-M2-0001 | Planchas: 1`, y el reto confirma una segunda orden con exactamente dos planchas cargadas mediante `Include`.

Conexión con el siguiente punto

Con la relación uno-a-muchos ya expresada de forma explícita, 2.4 añadirá las relaciones uno-a-uno del modelo. Hasta entonces, 2.3 no incorpora entidades ni configuraciones propias de esos puntos posteriores.

Código acumulativo completo del estado 2.3

El siguiente archivo es el `Program.cs` real del estado validable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

```

using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    public string Estado { get; set; } = "Pendiente";
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
}

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public stringCodigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    public string? Descripcion { get; set; }
}

public class EstadoOrden
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<OrdenFabricacion>(entity =>
        {
            entity.ToTable("OrdenesFabricacion");
            entity.HasKey(o => o.Id);
            entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);
            entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);
            entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");
            entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
        });
    }
}

```

```

        entity.Property(o => o.Observaciones).HasMaxLength(500);
    });
    modelBuilder.Entity<PlanchaAcero>(entity =>
    {
        entity.ToTable("PlanchasAcero");
        entity.HasKey(x => x.Id);
        entity.Property(x => x.Peso).HasPrecision(18, 3);
        entity.Property(x => x.Activa).HasDefaultValue(true);
        entity.HasOne(x => x.Orden)
            .WithMany(o => o.Planchas)
            .HasForeignKey(x => x.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });
    modelBuilder.Entity<Aleacion>(entity =>
    {
        entity.ToTable("Aleaciones");
        entity.HasKey(a => a.Id);
        entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
        entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
        entity.Property(a => a.Descripcion).HasMaxLength(500);
    });
    modelBuilder.Entity<EstadoOrden>(entity =>
    {
        entity.ToTable("EstadosOrden");
        entity.HasKey(e => e.Id);
        entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
        entity.Property(e => e.Descripcion).HasMaxLength(250);
        entity.Property(e => e.Activo).HasDefaultValue(true);
    });
}

}

public interface IOordenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    int Guardar();
}

public sealed class OrdenRepositorio : IOordenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) =>
        _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOordenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOordenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }
}

```



```

    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();
        services.AddDbContext<AceriaDbContext>(options =>
            options
                .UseSqlServer(connectionString, sql =>
                    {
                        sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                        sql.CommandTimeout(60);
                    }
                )
                .LogTo(global::System.Console.WriteLine, new[] {
                    "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)
                .EnableDetailedErrors());

        services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
        services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

        using var provider = services.BuildServiceProvider(new ServiceProviderOptions
        {
            ValidateScopes = true,
            ValidateOnBuild = true
        });

        using var scope = provider.CreateScope();
        var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
        context.Database.EnsureDeleted();
        context.Database.Migrate();

        var orden = new OrdenFabricacion
        {
            NumeroOrden = "OF-M2-0001",
            Cliente = "Constructora del Norte",
            FechaCreacion = DateTime.UtcNow,
            FechaEntrega = DateTime.Today.AddDays(14),
            Estado = "Pendiente",
            Observaciones = "Orden de validación M2",
            Planchas =
            {
                new PlanchaAcero
                {
                    Espesor = 10.5,
                    Ancho = 1500,
                    Largo = 3000,
                    Peso = 371.250m,
                    Activa = true,
                }
            },
        };

        context.OrdenesFabricacion.Add(orden);
        context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa",

```

```

Activo = true });
context.SaveChanges();

var cargada = context.OrdenesFabricacion.Include(o => o.Planchas).Single(o => o.NumeroOrden ==
"0F-M2-0001");
global::System.Console.WriteLine($"2.3 OK | {cargada.NumeroOrden} | Planchas:
{cargada.Planchas.Count}");
}
}

```

Explicación línea a línea del código acumulativo 2.3

Las referencias siguientes usan la numeración real de M02/PROYECTO/2.3/Program.cs. Se explican todas las líneas no vacías para que el documento sea trazable con el archivo que compila.

Línea 1: `using System.ComponentModel.DataAnnotations;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 2: `using System.ComponentModel.DataAnnotations.Schema;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 3: `using Microsoft.EntityFrameworkCore;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 4: `using Microsoft.Extensions.Configuration;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 5: `using Microsoft.Extensions.DependencyInjection;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 6: `using Microsoft.Extensions.Logging;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 8: `namespace AceriaData.ConsoleApp;` → declara el espacio de nombres del estado acumulativo.

Línea 10: `public class OrdenFabricacion` → declara la clase OrdenFabricacion que forma parte del estado acumulativo del ejercicio.

Línea 11: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 12: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 13: `public string NumeroOrden { get; set; } = string.Empty;` → declara la propiedad NumeroOrden de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 14: `public string Cliente { get; set; } = string.Empty;` → declara la propiedad Cliente de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 15: `public DateTime FechaCreacion { get; set; }` → declara la propiedad FechaCreacion de tipo DateTime que EF Core o la lógica de negocio utiliza en este modelo.

Línea 16: `public DateTime? FechaEntrega { get; set; }` → declara la propiedad FechaEntrega de tipo DateTime? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 17: `public string Estado { get; set; } = "Pendiente";` → declara la propiedad Estado de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 18: `public string? Observaciones { get; set; }` → declara la propiedad Observaciones de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 19: `public List<PlanchaAcero> Planchas { get; set; } = new();` → declara la propiedad Planchas de tipo List que EF Core o la lógica de negocio utiliza en este modelo.

Línea 20: `}` → cierra el bloque de código actual.

Línea 22: `public class PlanchaAcero` → declara la clase `PlanchaAcero` que forma parte del estado acumulativo del ejercicio.

Línea 23: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 24: `public int Id { get; set; }` → declara la propiedad `Id` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 25: `public int OrdenId { get; set; }` → declara la propiedad `OrdenId` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 26: `public double Espesor { get; set; }` → declara la propiedad `Espesor` de tipo `double` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 27: `public double Ancho { get; set; }` → declara la propiedad `Ancho` de tipo `double` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 28: `public double Largo { get; set; }` → declara la propiedad `Largo` de tipo `double` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 29: `public decimal Peso { get; set; }` → declara la propiedad `Peso` de tipo `decimal` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 30: `public bool Activa { get; set; } = true;` → declara la propiedad `Activa` de tipo `bool` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 31: `public OrdenFabricacion Orden { get; set; } = null!` → declara la propiedad `Orden` de tipo `OrdenFabricacion` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 32: `}` → cierra el bloque de código actual.

Línea 34: `public class Aleacion` → declara la clase `Aleacion` que forma parte del estado acumulativo del ejercicio.

Línea 35: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 36: `public int Id { get; set; }` → declara la propiedad `Id` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 37: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad `Nombre` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 38: `public string Codigo { get; set; } = string.Empty;` → declara la propiedad `Codigo` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 39: `public double PorcentajeCarbono { get; set; }` → declara la propiedad `PorcentajeCarbono` de tipo `double` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 40: `public double PorcentajeManganeso { get; set; }` → declara la propiedad `PorcentajeManganeso` de tipo `double` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 41: `public string? Descripcion { get; set; }` → declara la propiedad `Descripcion` de tipo `string?` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 42: `}` → cierra el bloque de código actual.

Línea 44: `public class EstadoOrden` → declara la clase `EstadoOrden` que forma parte del estado acumulativo del ejercicio.

Línea 45: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 46: `public int Id { get; set; }` → declara la propiedad `Id` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 47: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad `Nombre` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 48: `public string Descripcion { get; set; } = string.Empty;` → declara la propiedad Descripcion de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 49: `public bool Activo { get; set; } = true;` → declara la propiedad Activo de tipo bool que EF Core o la lógica de negocio utiliza en este modelo.

Línea 50: `}` → cierra el bloque de código actual.

Línea 52: `public class AceriaDbContext : DbContext` → declara la clase AceriaDbContext que forma parte del estado acumulativo del ejercicio.

Línea 53: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 54: `public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }` → declara el constructor o método AceriaDbContext con los parámetros necesarios para este componente.

Línea 56: `public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 57: `public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 58: `public DbSet<Aleacion> Aleaciones => Set<Aleacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 59: `public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 62: `protected override void OnModelCreating(ModelBuilder modelBuilder)` → sobrescribe OnModelCreating para definir configuración explícita del modelo.

Línea 63: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 64: `base.OnModelCreating(modelBuilder);` → sobrescribe OnModelCreating para definir configuración explícita del modelo.

Línea 65: `modelBuilder.Entity<OrdenFabricacion>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 66: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 67: `entity.ToTable("OrdenesFabricacion");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 68: `entity.HasKey(o => o.Id);` → define la clave primaria de la entidad.

Línea 69: `entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 70: `entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);` → selecciona una propiedad escalar para continuar su configuración.

Línea 71: `entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");` → selecciona una propiedad escalar para continuar su configuración.

Línea 72: `entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");` → selecciona una propiedad escalar para continuar su configuración.

Línea 73: `entity.Property(o => o.Observaciones).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 74: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 75: `modelBuilder.Entity<PlanchaAcero>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 76: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 77: `entity.ToTable("PlanchasAcero");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 78: `entity.HasKey(x => x.Id);` → define la clave primaria de la entidad.

Línea 79: `entity.Property(x => x.Peso).HasPrecision(18, 3);` → selecciona una propiedad escalar para continuar su configuración.

Línea 80: `entity.Property(x => x.Activa).HasDefaultValue(true);` → selecciona una propiedad escalar para continuar su configuración.

Línea 81: `entity.HasOne(x => x.Orden)` → inicia la configuración de una navegación de referencia.

Línea 82: `.WithMany(o => o.Planchas)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 83: `.HasForeignKey(x => x.OrdenId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 84: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 85: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 86: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 87: `modelBuilder.Entity<Aleacion>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 88: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 89: `entity.ToTable("Aleaciones");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 90: `entity.HasKey(a => a.Id);` → define la clave primaria de la entidad.

Línea 91: `entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);` → selecciona una propiedad escalar para continuar su configuración.

Línea 92: `entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);` → selecciona una propiedad escalar para continuar su configuración.

Línea 93: `entity.Property(a => a.Descripcion).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 94: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 95: `modelBuilder.Entity<EstadoOrden>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 96: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 97: `entity.ToTable("EstadosOrden");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 98: `entity.HasKey(e => e.Id);` → define la clave primaria de la entidad.

Línea 99: `entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 100: `entity.Property(e => e.Descripcion).HasMaxLength(250);` → selecciona una propiedad escalar para continuar su configuración.

Línea 101: `entity.Property(e => e.Activo).HasDefaultValue(true);` → selecciona una propiedad escalar para continuar su configuración.

Línea 102: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 105: `}` → cierra el bloque de código actual.

Línea 106: `}` → cierra el bloque de código actual.

Línea 108: `public interface IOrdenRepositorio` → declara el contrato `IOrdenRepositorio` para desacoplar consumidores e implementación.

Línea 109: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 110: `List<OrdenFabricacion> ObtenerTodas();` → declara en el contrato la operación que debe devolver todas las órdenes visibles para el repositorio.

Línea 111: `OrdenFabricacion? ObtenerPorId(int id);` → declara en el contrato la búsqueda de una orden por su clave primaria y permite devolver null si no existe.

Línea 112: `OrdenFabricacion? ObtenerPorNumero(string numeroOrden);` → declara en el contrato la búsqueda por el identificador de negocio `NumeroOrden`.

Línea 113: `void Agregar(OrdenFabricacion orden);` → declara la operación que incorpora una orden al repositorio para que EF Core la siga como nueva.

Línea 114: `void Eliminar(OrdenFabricacion orden);` → declara la operación que marca una orden para eliminación física cuando el escenario lo requiere.

Línea 115: `int Guardar();` → declara la operación que confirma los cambios pendientes y devuelve el número de entradas afectadas.

Línea 116: `}` → cierra el bloque de código actual.

Línea 118: `public sealed class OrdenRepositorio : IOrdenRepositorio` → declara la clase `OrdenRepositorio` que forma parte del estado acumulativo del ejercicio.

Línea 119: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 120: `private readonly AceriaDbContext _context;` → declara el campo privado `_context` de tipo `AceriaDbContext` que conserva una dependencia de la clase.

Línea 121: `public OrdenRepositorio(AceriaDbContext context) => _context = context;` → declara el constructor o método `OrdenRepositorio` con los parámetros necesarios para este componente.

Línea 123: `public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();` → ordena el resultado de la consulta antes de materializarlo.

Línea 124: `public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);` → implementa la búsqueda por clave primaria mediante `Find`, aprovechando primero el `ChangeTracker` y después la base de datos.

Línea 125: `public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);` → recupera la primera coincidencia o null cuando no existe.

Línea 126: `public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 127: `public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);` → marca la entidad indicada para borrado al confirmar los cambios.

Línea 128: `public int Guardar() => _context.SaveChanges();` → persiste en SQL Server los cambios seguidos por el `DbContext`.

Línea 129: `}` → cierra el bloque de código actual.

Línea 131: `public interface IServicioOrdenes` → declara el contrato `IServicioOrdenes` para desacoplar consumidores e implementación.

Línea 132: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 133: `string ObtenerResumen(int ordenId);` → declara el servicio que construye un resumen de una orden a partir de su identificador.

Línea 134: `}` → cierra el bloque de código actual.

Línea 136: `public sealed class ServicioOrdenes : IServicioOrdenes` → declara la clase `ServicioOrdenes` que forma parte del estado acumulativo del ejercicio.

Línea 137: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 138: `private readonly IOrdenRepositorio _repositorio;` → declara el campo privado `_repositorio` de tipo `IOrdenRepositorio` que conserva una dependencia de la clase.

Línea 139: `private readonly AceriaDbContext _context;` → declara el campo privado `_context` de tipo `AceriaDbContext` que conserva una dependencia de la clase.

Línea 140: `public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)` → declara el constructor o método `ServicioOrdenes` con los parámetros necesarios para este componente.

Línea 141: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 142: `_repositorio = repositorio;` → guarda en el servicio el repositorio inyectado para reutilizarlo en sus operaciones.

Línea 143: `_context = context;` → guarda el `DbContext` inyectado para consultar los datos necesarios durante el servicio.

Línea 144: `}` → cierra el bloque de código actual.

Línea 146: `public string ObtenerResumen(int ordenId)` → declara el método que obtiene la orden y compone el resumen solicitado por el servicio.

Línea 147: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 148: `var orden = _repositorio.ObtenerPorId(ordenId);` → consulta la orden mediante el repositorio y conserva el resultado anulable en una variable local.

Línea 149: `if (orden is null) return $"Orden {ordenId} no encontrada";` → evalúa la condición y ejecuta el bloque asociado sólo cuando se cumple.

Línea 150: `var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);` → cuenta las filas que cumplen la consulta construida.

Línea 151: `return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";` → devuelve el resultado calculado al código que invocó el método.

Línea 152: `}` → cierra el bloque de código actual.

Línea 153: `}` → cierra el bloque de código actual.

Línea 155: `public static class Program` → declara la clase `Program` que forma parte del estado acumulativo del ejercicio.

Línea 156: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 157: `public static void Main()` → declara el punto de entrada ejecutable de la aplicación de consola.

Línea 158: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 159: `var configuration = new ConfigurationBuilder()` → inicia la construcción de la configuración de la aplicación.

Línea 160: `.SetBasePath(AppContext.BaseDirectory)` → establece la ruta base usada para localizar los archivos de configuración.

Línea 161: `.AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)` → incorpora `appsettings.json` a la configuración.

Línea 162: `.AddEnvironmentVariables()` → incorpora variables de entorno como fuente adicional de configuración.

Línea 163: `.Build();` → materializa la configuración o el objeto encadenado construido en las líneas anteriores.

Línea 165: `var connectionString = configuration.GetConnectionString("AceriaDB")` → obtiene la cadena de conexión `AceriaDB` desde la configuración.

Línea 166: `?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");` → interrumpe el arranque con un error explícito si la configuración no contiene `ConnectionStrings:AceriaDB`.

Línea 168: `var services = new ServiceCollection();` → crea la colección donde se registran las dependencias de la aplicación.

Línea 169: `services.AddDbContext<AceriaDbContext>(options =>` → registra el `DbContext` en el contenedor de inyección de dependencias.

Línea 170: `options` → continúa la configuración del `DbContext` sobre el parámetro `options` recibido por la lambda de `AddDbContext`.

Línea 171: `.UseSqlServer(connectionString, sql =>` → selecciona `SQL Server` como proveedor de `EF Core`.

Línea 172: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 173: `sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);` → habilita reintentos ante fallos transitorios del proveedor `SQL Server`.

Línea 174: `sql.CommandTimeout(60);` → establece el tiempo máximo permitido para los comandos `SQL`.

Línea 175: `})` → cierra la lambda de configuración actual y devuelve el control a la llamada encadenada que la contiene.

Línea 176: `.LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)` → configura la salida del logging de `EF Core`.

Línea 177: `.EnableDetailedErrors();` → activa errores detallados para facilitar el diagnóstico del laboratorio.

Línea 179: `services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();` → registra un servicio `scoped` para compartir dependencias dentro del mismo ámbito.

Línea 180: `services.AddScoped<IServicioOrdenes, ServicioOrdenes>();` → registra un servicio `scoped` para compartir dependencias dentro del mismo ámbito.

Línea 182: `using var provider = services.BuildServiceProvider(new ServiceProviderOptions` → construye el `ServiceProvider` con validación de servicios y garantiza su liberación al salir del ámbito mediante `using var`.

Línea 183: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 184: `ValidateScopes = true,` → activa la validación de ámbitos para detectar dependencias `scoped` usadas desde ámbitos incorrectos.

Línea 185: `ValidateOnBuild = true` → obliga al contenedor a validar las resoluciones registradas al construir el `ServiceProvider`.

Línea 186: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 188: `using var scope = provider.CreateScope();` → crea un ámbito de DI para resolver servicios scoped y lo libera automáticamente al terminar el bloque.

Línea 189: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → resuelve un servicio obligatorio desde el contenedor de dependencias.

Línea 190: `context.Database.EnsureDeleted();` → elimina la base del laboratorio para reproducir el escenario desde un estado limpio.

Línea 191: `context.Database.Migrate();` → aplica las migraciones pendientes y deja AceriaDB en el esquema del punto.

Línea 193: `var orden = new OrdenFabricacion` → crea la entidad OrdenFabricacion que concentra el grafo de datos del escenario acumulativo.

Línea 194: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 195: `NumeroOrden = "0F-M2-0001",` → asigna el identificador de negocio usado por la orden de validación acumulativa del módulo.

Línea 196: `Cliente = "Constructora del Norte",` → asigna el cliente de la orden utilizada en el escenario E2E del punto.

Línea 197: `FechaCreacion = DateTime.UtcNow,` → asigna a la orden la fecha de creación en UTC para el escenario reproducible del punto.

Línea 198: `FechaEntrega = DateTime.Today.AddDays(14),` → establece una fecha de entrega prevista catorce días después de la fecha local actual.

Línea 199: `Estado = "Pendiente",` → inicializa la orden con el estado Pendiente, coherente con el flujo del laboratorio.

Línea 200: `Observaciones = "Orden de validación M2",` → añade texto opcional para comprobar el mapeo de Observaciones.

Línea 201: `Planchas =` → inicia el inicializador de la colección de planchas asociadas a la orden.

Línea 202: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 203: `new PlanchaAcero` → crea una nueva plancha dentro del inicializador de la colección Planchas de la orden.

Línea 204: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 205: `Espesor = 10.5,` → asigna a la plancha un espesor positivo para satisfacer las reglas del modelo.

Línea 206: `Ancho = 1500,` → asigna el ancho de la plancha usada en el escenario de persistencia.

Línea 207: `Largo = 3000,` → asigna el largo de la plancha usada en el escenario de persistencia.

Línea 208: `Peso = 371.250m,` → asigna un peso decimal con tres posiciones para validar la precisión configurada.

Línea 209: `Activa = true,` → marca la plancha como activa para que sea visible en los estados que aplican ese criterio.

Línea 210: `}` → cierra el bloque de código actual.

Línea 211: `},` → continúa el inicializador o la llamada iniciada en el bloque actual.

Línea 212: `};` → cierra el inicializador del objeto y finaliza la instrucción.

Línea 214: `context.OrdenesFabricacion.Add(orden);` → registra una entidad o relación nueva en el ChangeTracker para insertarla al guardar.

Línea 215: `context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo = true });` → registra una entidad o relación nueva en el ChangeTracker para insertarla al guardar.

Línea 216: `context.SaveChanges();` → persiste en SQL Server los cambios seguidos por el DbContext.

Línea 218: `var cargada = context OrdenesFabricacion.Include(o => o.Planchas).Single(o => o.NumeroOrden == "0F-M2-0001");` → incluye de forma eager la navegación relacionada en la consulta.

Línea 219: `global::System.Console.WriteLine($"2.3 OK | {cargada.NumeroOrden} | Planchas: {cargada.Planchas.Count}");` → escribe en consola la evidencia usada para verificar el comportamiento del estado.

Línea 220: `}` → cierra el bloque de código actual.

Línea 221: `}` → cierra el bloque de código actual.